Dr. D. Y. Patil Technical Campus Varale, Talegaon, Pune
(Savitribai Phule Pune University)
[Institute Code: 6991]



A Report on

"FreshFarma – Farm to Society"

SUBMITTED BY:

Swaroop Pawar – SE0A25160 | Divya Kalwade – SE0A25129 | Shivam Sonawane – SE0A25171

Under the Guidance of

Prof. Sayali Pawar

DEPARTMENT OF ELECTRONIC & COMPUTER ENGINEERING

(Academic Year: 2026-27)

ABSTRACT

Abstract

Fresh Farma is a digital storefront concept designed to make browsing and comparing fruits and vegetables easier and more convenient. As described in the supplied presentation, the platform includes category-based browsing, product cards, pricing, customer ratings, stock availability, harvest or supplier details, and an interactive basket feature. These features help users explore products, compare prices, check availability, and select items according to their preferences.

This report also presents a supplementary C++ coursework model that demonstrates object-oriented programming and basic data structures. Each product is represented as an object containing attributes such as name, category, and price, with multiple products stored in a collection. Linear Search is used to find products by name, while Bubble Sort arranges products according to their prices. This model connects theoretical programming concepts with a practical agricultural e-commerce scenario, demonstrating how basic algorithms can support product management in a digital storefront.

PROBLEM STATEMENT

Background

Discovering and comparing produce can require visiting multiple stalls or stores. Product availability and displayed information may vary by location and time. A digital storefront can make product discovery and comparison easier by presenting produce details in one place.

Problem statement

To present fruits and vegetables in an organized digital catalogue and demonstrate, through a small C++ model, how product records can be stored, displayed, searched by name, and sorted by price.

Objectives

1. Organize produce as structured product records.
2. Display the product name, category, and price clearly.
3. Search for a product using its name.
4. Sort products by price in ascending or descending order.
5. Demonstrate object-oriented programming, Linear Search, and Bubble Sort.
6. Keep the academic model simple enough to understand and test.

Scope and limitation

The supplied presentation describes a storefront experience. It explicitly states that backend, database, payment, delivery, and inventory operations are not publicly verified. Therefore, this report does not claim that those services are implemented or connected.

DATA STRUCTURES USED AND JUSTIFICATION

1. Product class (record)

The Product class groups a product's name, category, and price. Keeping these fields together prevents the program from treating the name and price as unrelated values. A constructor initializes each product, while getter functions provide access to its details.

2. Vector collection

A vector<Product> stores the catalogue as one ordered collection. It supports sequential traversal for display and search, and it allows products to be swapped during sorting. For a small coursework example, this structure is easy to read and extend.

3. Linear Search

Linear Search examines products from the first element onward until the requested name is found or the collection ends. Its worst-case time complexity is O(n), where n is the number of products. It is straightforward and suitable for this small demonstration.

4. Bubble Sort

Bubble Sort compares neighbouring product prices and swaps adjacent products when they are in the wrong order. The program supports ascending and descending order. Its worst-case time complexity is $O(n^2)$, and its best case with the early-stop optimization is O(n) when the list is already sorted.

5. Object-oriented concepts

Encapsulation is demonstrated by private product fields. The constructor sets initial values. Operator overloading defines a price-based comparison between Product objects. The sorting function uses explicit price comparisons so the chosen direction is clear.

Justification

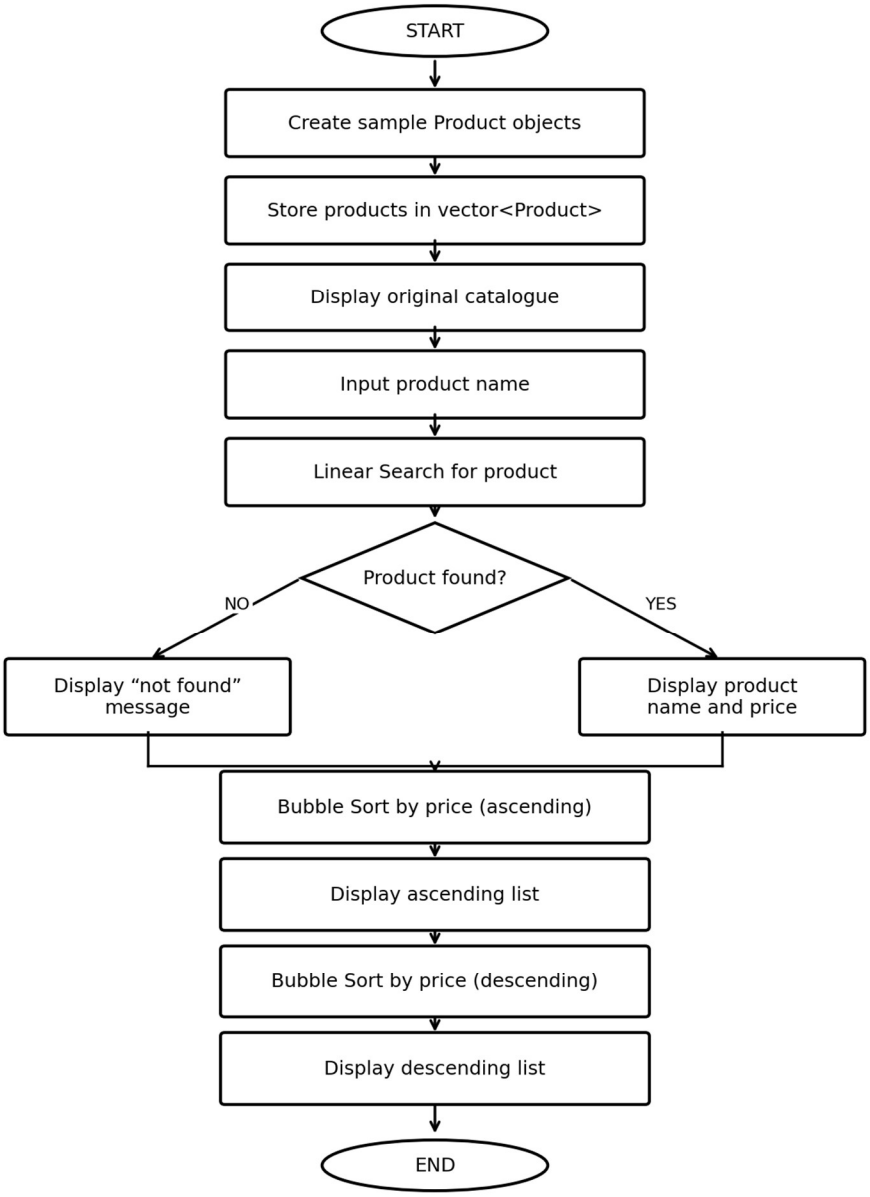
These choices prioritize clarity and demonstration of core DSA concepts rather than performance at large scale. For a much larger catalogue, a more efficient sorting algorithm and an indexed search approach could be considered.

FLOWCHART / CLASS DIAGRAM SYSTEM DESIGN /

System overview

Input: a fixed sample catalogue and a product name entered by the user. Processing: display records, perform Linear Search, and sort records with Bubble Sort. Output: search result plus ascending and descending price lists.

Flowchart (text form)



Class diagram (text form)

```
Product
 ├─ Private data: name : string
 ├─ Private data: category : string
 ├─ Private data: price : double
 ├─ Constructor: Product(name, category, price)
 ├─ getName() : string
 ├─ getCategory() : string
 ├─ getPrice() : double
 └─ operator<(other) : bool

Catalogue operations
 ├─ displayProducts(vector<Product>)
 ├─ linearSearch(vector<Product>, target) : int
 └─ bubbleSortByPrice(vector<Product>, ascending) : void
```

Design note

This is a design for the supplementary C++ model, not a reverse-engineered diagram of the website's internal architecture.

COMPLETE C++ SOURCE CODE

```
#include <iostream>
#include <vector>
#include <string>
#include <iomanip>
#include <algorithm>
using namespace std;

// Represents one fruit or vegetable in the FreshFarma catalogue.
class Product {
private:
    string name;
    string category;
    double price;  // Price per kg in Indian Rupees (sample data)

public:
    // Constructor initializes a product record.
    Product(string productName, string productCategory, double productPrice)
        : name(productName), category(productCategory), price(productPrice) {}

    // Getter functions provide read-only access to private data.
    string getName() const { return name; }
    string getCategory() const { return category; }
    double getPrice() const { return price; }

    // Overload < so products can be compared by price.
    bool operator<(const Product& other) const {
        return price < other.price;
    }
};

// Displays all products in a consistent tabular format.
void displayProducts(const vector<Product>& products) {
    cout << left << setw(16) << "Product"
         << setw(16) << "Category"
         << "Price (Rs/kg)" << '\\n';
    cout << string(45, '-') << '\\n';

    for (const Product& product : products) {
        cout << left << setw(16) << product.getName()
             << setw(16) << product.getCategory()
             << fixed << setprecision(2) << product.getPrice()
             << '\\n';
    }
}

// Linear Search: checks each product name until a match is found.
int linearSearch(const vector<Product>& products, const string& target) {
    for (size_t i = 0; i < products.size(); ++i) {
        if (products[i].getName() == target) {
            return static_cast<int>(i); // Return zero-based position
        }
    }
    return -1; // -1 indicates that the product was not found
}

// Bubble Sort in ascending order of price.
// Neighbouring products are swapped when the left price is greater.
void bubbleSortByPrice(vector<Product>& products, bool ascending = true) {
    const size_t n = products.size();

    for (size_t i = 0; i < n; ++i) {
        bool swapped = false;

        for (size_t j = 0; j + 1 < n - i; ++j) {
            bool shouldSwap;

            if (ascending) {
```

```
                shouldSwap = products[j].getPrice() >
                             products[j + 1].getPrice();
            } else {
                shouldSwap = products[j].getPrice() <
                             products[j + 1].getPrice();
            }

            if (shouldSwap) {
                swap(products[j], products[j + 1]);
                swapped = true;
            }
        }

        // If a pass makes no swaps, the list is already sorted.
        if (!swapped) {
            break;
        }
    }
}

int main() {
    // Illustrative sample values from the project presentation.
    // These are academic demo values, not verified live website prices.
    vector<Product> products = {
        Product("Mango",  "Fruit",     120.00),
        Product("Apple",  "Fruit",     180.00),
        Product("Potato", "Vegetable",  30.00),
        Product("Tomato", "Vegetable",  40.00)
    };

    cout << "FRESHFARMA - FARM TO SOCIETY\\n";
    cout << "Initial product catalogue\\n";
    displayProducts(products);

    // Search for a product by its name.
    string target;
    cout << "\\nEnter product name to search (example: Tomato): ";
    getline(cin, target);

    int position = linearSearch(products, target);
    if (position != -1) {
        cout << target << " found. Price: Rs "
             << fixed << setprecision(2)
             << products[position].getPrice() << " per kg.\\n";
    } else {
        cout << target << " was not found in the catalogue.\\n";
    }

    // Sort by ascending price and display the result.
    bubbleSortByPrice(products, true);
    cout << "\\nProducts sorted by price (low to high)\\n";
    displayProducts(products);

    // Optional descending sort demonstration.
    bubbleSortByPrice(products, false);
    cout << "\\nProducts sorted by price (high to low)\\n";
    displayProducts(products);

    return 0;
}
```

SAMPLE INPUT / OUTPUT

Sample catalogue

The following values are illustrative academic sample data from the presentation, not verified live website prices.

Initial data

Product	Category	Price (Rs/kg)
Mango	Fruit	120.00
Apple	Fruit	180.00
Potato	Vegetable	30.00
Tomato	Vegetable	40.00

Sample search input

Tomato

Expected search result

Tomato found. Price: Rs 40.00 per kg.

Expected ascending order

Potato – Rs 30.00

Tomato – Rs 40.00

Mango – Rs 120.00

Apple – Rs 180.00

Expected descending order

Apple – Rs 180.00

Mango – Rs 120.00

Tomato – Rs 40.00

Potato – Rs 30.00

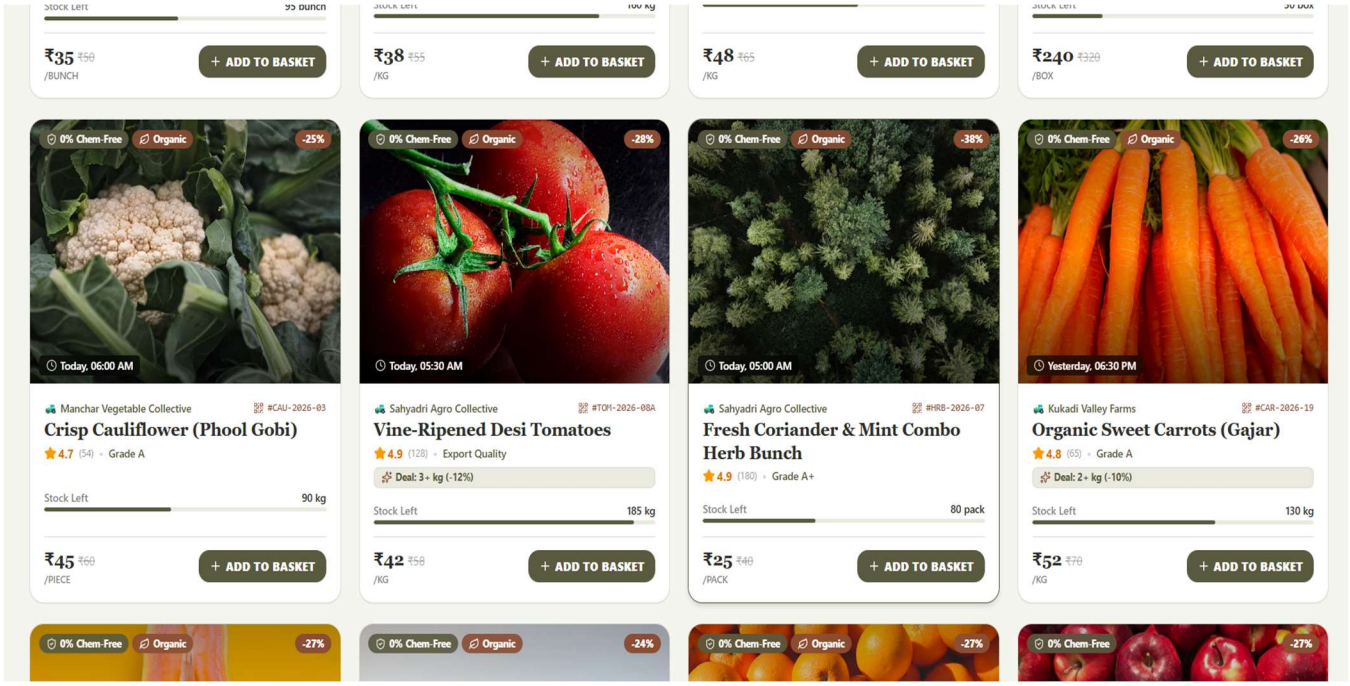
Additional test

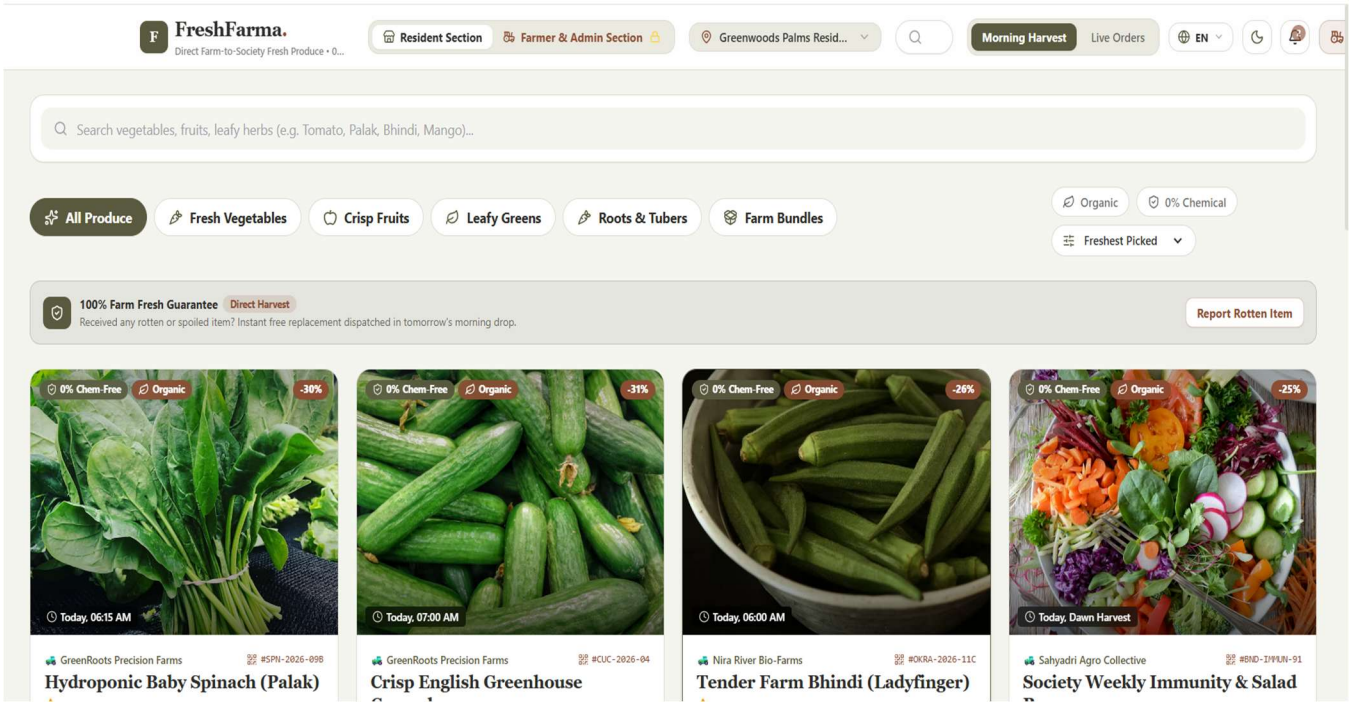
Search input: Banana

Expected result: Banana was not found in the catalogue.

Test summary

Display catalogue → four product records appear. Search Tomato → found at its collection position. Sort ascending → Potato, Tomato, Mango, Apple. Search Banana → not-found message.


₹35 ₹50 /BUNCH
+ ADD TO BASKET
₹38 ₹55 /KG
+ ADD TO BASKET
₹48 ₹65 /KG
+ ADD TO BASKET
₹240 ₹320 /BOX
+ ADD TO BASKET
0% Chem-Free
Organic
-25%
Today, 06:00 AM
Manchar Vegetable Collective
#CAU-2026-03
Crisp Cauliflower (Phool Gobi)
4.7 (54) • Grade A
Stock Left
90 kg
₹45 ₹60 /PIECE
+ ADD TO BASKET
-28%
Today, 05:30 AM
Sahyadri Agro Collective
#TOM-2026-08A
Vine-Ripened Desi Tomatoes
4.9 (128) • Export Quality
Deal: 3+ kg (-12%)
Stock Left
185 kg
₹42 ₹58 /KG
+ ADD TO BASKET
-38%
Today, 05:00 AM
Sahyadri Agro Collective
#HRB-2026-07
Fresh Coriander & Mint Combo Herb Bunch
4.9 (180) • Grade A+
Stock Left
80 pack
₹25 ₹40 /PACK
+ ADD TO BASKET
-26%
Yesterday, 06:30 PM
Kukadi Valley Farms
#CAR-2026-19
Organic Sweet Carrots (Gajar)
4.8 (65) • Grade A
Deal: 2+ kg (-10%)
Stock Left
130 kg
₹52 ₹70 /KG
+ ADD TO BASKET
-27%
-24%
-27%
-27%


FreshFarma.
Direct Farm-to-Society Fresh Produce • 0...
Resident Section
Farmer & Admin Section
Greenwoods Palms Resid...
Morning Harvest
Live Orders
EN
Search vegetables, fruits, leafy herbs (e.g. Tomato, Palak, Bhindi, Mango)...
All Produce
Fresh Vegetables
Crisp Fruits
Leafy Greens
Roots & Tubers
Farm Bundles
Organic
0% Chemical
Freshest Picked
100% Farm Fresh Guarantee
Direct Harvest
Received any rotten or spoiled item? Instant free replacement dispatched in tomorrow's morning drop.
Report Rotten Item
0% Chem-Free
Organic
-30%
Today, 06:15 AM
GreenRoots Precision Farms
#SPN-2026-09B
Hydroponic Baby Spinach (Palak)
-31%
Today, 07:00 AM
GreenRoots Precision Farms
#CUC-2026-04
Crisp English Greenhouse
-26%
Today, 06:00 AM
Nira River Bio-Farms
#OKRA-2026-11C
Tender Farm Bhindi (Ladyfinger)
-25%
Today, Dawn Harvest
Sahyadri Agro Collective
#BND-IMMUN-91
Society Weekly Immunity & Salad

CONCLUSION AND FUTURE SCOPE-

Conclusion

FreshFarma presents a practical use case for organizing fruit and vegetable information in a digital catalogue. The supplementary C++ model demonstrates how a Product class can group related fields, how a vector can hold a collection of products, how Linear Search can find an item by name, and how Bubble Sort can order products by price. The example is intentionally small and transparent so its operations can be followed and tested.

Future scope

1. Add more fruits and vegetables and validate input values.
2. Add category-based filtering and case-insensitive search.
3. Store product data in a persistent database if a verified backend is implemented.
4. Add stock updates, authentication, payment, order tracking, and delivery only when those services are designed and tested.
5. For larger catalogues, use a more efficient sorting algorithm such as Merge Sort or std::sort.
6. Add automated test cases for empty lists, duplicate names, equal prices, and missing products.
7. Integrate the academic model with a real application only after defining and verifying an API and data source.

References / source basis

Primary source: supplied PowerPoint, “FreshFarma – Farm to Society: An E-Commerce Platform for Fruits and Vegetables” (Data Structure and Algorithm Course Project, academic year 2026–2027).

Live storefront cited in the presentation: https://freshfarma.vercel.app/

Note: sample values and the C++ implementation in this report are illustrative coursework material, not verified live website data or source code.